

2 Name your **Jupyter Notebook** as:
TASK2_<your class>_<your name>.ipynb

A programmer is writing Coconut Island Game to be played on the computer.

The island is represented as a rectangular grid, 10 rows by 15 columns. Each square of land on the island is represented by a pair of coordinates in a 2D array. The top left square of the island has coordinates where row = 0 and column = 0. There are 10 squares of land down and 15 squares of land across.

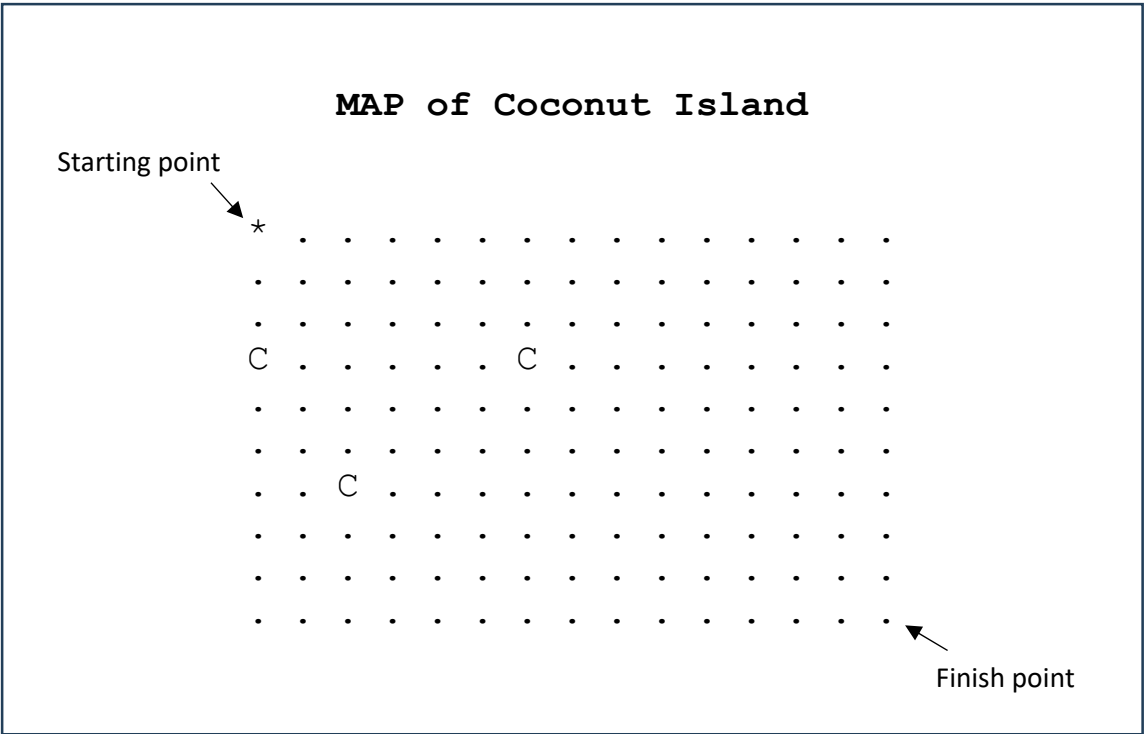
The computer will:

- generate three random locations where coconut trees will be located
- start player at the top left corner of the grid (row = 0, column = 0)
- prompt the player for the move, in the format direction left (L), right (R), up (U), down (D), followed by a space and the distance representing the number of grids moved in the chosen direction (e.g.: R 6 would move the player six grids to the right from the current location)
- plot the path the player has moved
- display the contents of the array by outputting for each square of land:
 - '.' for unexplored land
 - 'C' for coconut tree
 - 'E' for eaten coconut when player's move ends on a coconut tree
 - '*' for explored land i.e. path walked on by player

The rule of the game is:

- player has maximum eight moves
- to win the game, the player needs to
 - visit the three squares with coconut trees to eat them, and
 - reach the finish point at bottom right corner (row = 9, column = 14)

A sample grid of the island, with three coconut trees, is displayed:



[JPJC/2024/Q2 Cont']

The following is a sample run of the game:

```

Welcome to Coconut Island Game!

*
. . . . .
. . . . .
. . . . .
C . . . . C . . . .
. . . . .
. . . . .
. . C . . . . .
. . . . .
. . . . .
. . . . .

You are at position: (0, 0) Coconut eaten: 0
Enter your move (L/R/U/D distance): D 3

```

```

* . . . . . . . . . . . .
* . . . . . . . . . . . .
* . . . . . . . . . . . .
E . . . . C . . . . . . .
. . . . . . . . . . . .
. . . . . . . . . . . .
. . C . . . . . . . . .
. . . . . . . . . . . .
. . . . . . . . . . . .
. . . . . . . . . . . .

```

You are at position: (3, 0) Coconut eaten: 1
Enter your move (L/R/U/D distance): **R 6**

```

*   .   .   .   .   .   .   .   .   .   .   .   .   .   .
*   .   .   .   .   .   .   .   .   .   .   .   .   .
*   .   .   .   .   .   .   .   .   .   .   .   .   .
E * * * * * E   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .   .   .   .
.   .   C   .   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .   .   .   .
.   .   .   .   .   .   .   .   .   .   .   .   .

```

You are at position: (3, 6) Coconut eaten: 2
Enter your move (L/R/U/D distance): **D 3**

```

*   .   .   .   .   .   .   .   .   .   .   .   .   .   .
*   .   .   .   .   .   .   .   .   .   .   .   .   .
*   .   .   .   .   .   .   .   .   .   .   .   .   .
E * * * * * E   .   .   .   .   .   .   .   .   .
    .   .   .   .   .   .   .   .   .   .   .   .
    .   .   .   .   .   .   .   .   .   .   .   .
    .   .   C   .   .   .   *   .   .   .   .   .   .
    .   .   .   .   .   .   .   .   .   .   .   .
    .   .   .   .   .   .   .   .   .   .   .   .
    .   .   .   .   .   .   .   .   .   .   .   .
    .   .   .   .   .   .   .   .   .   .   .   .
You are at position: (6, 6)  Coconut eaten: 2
Enter your move (L/R/U/D distance): L 4

```

[JPJC/2024/Q2 Cont']

You are at position: (6, 2) Coconut eaten: 3
Enter your move (L/R/U/D distance): **D 3**

A grid of dots arranged in 6 rows and 10 columns. The horizontal axis is labeled 'x' at the far right, and the vertical axis is labeled 'y' at the top. A single star symbol (*) is placed at the intersection of the second column from the left and the third row from the bottom.

You are at position: (9, 2) Coconut eaten: 3
Enter your move (L/R/U/D distance): **R 12**

You are at position: (9, 14) Coconut eaten: 3

You have won!

You have reached finish point and eaten 3 coconuts.

[JPJC/2024/Q2 Cont']

The Coconut Island Game is to be implemented using procedural programming.

Task 2.1

Write the following functions:

1. `createIsland(rowsize, colsize)`
 - creates a `grid` of size `rowsize` x `colsize`, its elements initialized to `'.'`, and with the starting position marked by `'*'` at the top-left corner (`row = 0`, `col = 0`)
 - returns the `grid`
2. `plantCoconuts(grid)`
 - plants 3 coconut trees `'C'`, randomly generated, on the `grid`
 - ensure the square is empty before planting
 - returns the `grid`
3. `display(grid)`
 - displays the current state of the `grid`
4. `move(grid, current, direction, distance, coconut)`
 - moves the player in the specified `direction` by a certain `distance` from the `current` position. The directions can be left (L), right (R), up (U), or down (D)
 - mark in the `grid` every square of the player's path with `'*'`, if the square does not have a coconut tree on it. If there is a coconut tree on the square, leave the marking `'C'` unchanged
 - if the player's move ends on a coconut tree, update the `grid` to mark the square as eaten `'E'` and increment `coconut eaten` by 1
 - returns a tuple containing the `grid`, `current` and `coconut`

Assume all moves the player input are valid which will not result in out-of-range coordinates.

[JPJC/2024/Q2 Cont']

Task 2.2

Write the function `main()` that uses the subroutines written in **Task 2.1** to perform the following:

- Create the island, called `grid`.
- Plant three coconut trees on the island.
- Initialize the player starting position as (row = 0, col = 0), coconut eaten as 0, and other necessary variables.
- Display the `grid`.
- Use a loop to do the following (maximum 8 moves):
 - Print player's current location and coconut eaten
 - Prompt player to input the move in the format `<direction> <space> <distance>`, where `direction` is a character L/R/U/D and `distance` is an integer. Refer to sample run.
 - Update the `grid` with the player's move.
 - Display the `grid`.
 - Check if player has won (reached finish point and eaten 3 coconuts). If yes, exit the loop.
- If player has won, display:
`You have won!`
`You have reached finish point and eaten 3 coconuts.`
- Otherwise, display the message:
`Sorry you did not win.`
`You did not reach the finish point.`
`and/or: You have not eaten enough coconut.`
- Exit game.

[10]

Run the `main()` function and show a game with a win or lose result.

[1]

Save your Jupyter Notebook for Task 2.